

// Jivops

Guía completa — 30 lecciones

Arquitectura · Traffic management · Seguridad · Observabilidad ·
Resiliencia y escalado · Operación avanzada

Contenido

Módulo 1: Fundamentos y arquitectura

- Lección 01 — Qué es un service mesh y por qué Istio
- Lección 02 — Arquitectura de Istio — control plane y data plane
- Lección 03 — Instalación de Istio (istioctl, profiles)
- Lección 04 — Sidecar injection — automática vs manual
- Lección 05 — Istio Ambient Mesh vs Sidecar mode

Módulo 2: Traffic management

- Lección 06 — VirtualService — enrutamiento de tráfico
- Lección 07 — DestinationRule — subsets y políticas
- Lección 08 — Gateway — entrada de tráfico al mesh
- Lección 09 — Canary releases con Istio
- Lección 10 — Circuit breaking y retries

Módulo 3: Seguridad

- Lección 11 — mTLS automático entre servicios
- Lección 12 — PeerAuthentication — políticas de mTLS
- Lección 13 — AuthorizationPolicy — control de acceso
- Lección 14 — RequestAuthentication — JWT validation
- Lección 15 — Gestión de certificados con Istio CA

Módulo 4: Observabilidad

- Lección 16 — Métricas automáticas con Istio (Prometheus)
- Lección 17 — Distributed tracing con Istio
- Lección 18 — Kiali — visualización del mesh
- Lección 19 — Access logs de Envoy
- Lección 20 — Dashboards de Grafana para Istio

Módulo 5: Resiliencia y escalado

- Lección 21 — Timeouts y retries avanzados
- Lección 22 — Fault injection — chaos testing
- Lección 23 — Rate limiting con Istio
- Lección 24 — Multi-cluster mesh
- Lección 25 — Escalado del control plane (istiod)

Módulo 6: Operación avanzada

Lección 26 — Actualización de Istio sin downtime

Lección 27 — Troubleshooting de sidecars y Envoy

Lección 28 — Rendimiento — overhead del sidecar

Lección 29 — Migración progresiva a Istio (brownfield)

Lección 30 — Proyecto final: service mesh completo listo para producción

Módulo 1: Fundamentos y arquitectura

Lección 01 — Qué es un service mesh y por qué Istio

Istio gestiona centralizadamente seguridad, observabilidad y tráfico entre servicios mediante sidecars, evitando repetir esa lógica en cada microservicio.

Lección 02 — Arquitectura de Istio — control plane y data plane

istiod (control plane) configura políticas; los proxies Envoy (data plane) procesan el tráfico real, escalando cada uno según necesidades distintas.

Lección 03 — Instalación de Istio (istioctl, profiles)

El perfil 'demo' activa componentes pensados para explorar funcionalidades, no para una instalación optimizada de producción real.

```
istioctl install --set profile=production
```

Lección 04 — Sidecar injection — automática vs manual

La inyección automática evita que cada desarrollador tenga que recordar añadir el sidecar manualmente a cada manifiesto, reduciendo olvidos.

```
istio-injection=enabled
```

Lección 05 — Istio Ambient Mesh vs Sidecar mode

Ambient Mesh comparte un proxy eficiente a nivel de nodo en vez de multiplicar el overhead de un sidecar completo por cada Pod individual.

Módulo 2: Traffic management

Lección 06 — VirtualService — enrutamiento de tráfico

El enrutamiento vive en un recurso declarativo separado, permitiendo cambiar el comportamiento de tráfico sin reconstruir ni redespregar la aplicación.

Lección 07 — DestinationRule — subsets y políticas

Los subsets identifican versiones mediante labels — requisito previo para que Istio sepa cómo dirigir tráfico entre v1 y v2 de un servicio.

Lección 08 — Gateway — entrada de tráfico al mesh

Gateway configura la entrada de tráfico externo; VirtualService define el enrutamiento interno — separarlos da flexibilidad para gestionar cada uno de forma independiente.

Lección 09 — Canary releases con Istio

Definir pesos entre subsets (ej. 90/10) da control preciso del porcentaje de tráfico, independiente del número de réplicas de cada versión.

```
weight: 10
```

Lección 10 — Circuit breaking y retries

Un circuit breaker deja de enviar tráfico a instancias fallando, dándoles margen para recuperarse en vez de seguir empeorando la situación.

```
outlierDetection: consecutiveErrors: 5
```

Módulo 3: Seguridad

Lección 11 — mTLS automático entre servicios

Istio cifra y autentica el tráfico entre servicios de forma transparente en el sidecar, sin que cada equipo implemente manejo de certificados por su cuenta.

Lección 12 — PeerAuthentication — políticas de mTLS

El modo PERMISSIVE acepta tráfico mTLS y texto plano a la vez, dando margen para migrar servicios gradualmente sin romper los que aún no tienen sidecar.

```
mtls: mode: STRICT
```

Lección 13 — AuthorizationPolicy — control de acceso

Permitir que solo 'frontend' llame a 'pagos' aplica menor privilegio a nivel de red interna, reduciendo el riesgo de movimiento lateral ante un compromiso.

Lección 14 — RequestAuthentication — JWT validation

Validar el JWT a nivel de sidecar garantiza que todos los servicios apliquen la misma validación, en vez de lógica propia potencialmente distinta por equipo.

Lección 15 — Gestión de certificados con Istio CA

La rotación automática y frecuente de certificados reduce la ventana de riesgo: si uno se filtrara, sería útil por mucho menos tiempo que uno de larga duración.

Módulo 4: Observabilidad

Lección 16 — Métricas automáticas con Istio (Prometheus)

Istio genera métricas a nivel de sidecar sin depender de que cada equipo instrumente su servicio, dando cobertura uniforme en todo el mesh.

Lección 17 — Distributed tracing con Istio

Aunque el sidecar genera spans automáticamente, la aplicación debe propagar los headers de tracing en sus propias llamadas internas para mantener la cadena completa.

Lección 18 — Kiali — visualización del mesh

El mapa visual de tráfico permite identificar de un vistazo qué conexión específica muestra errores, sin revisar cada servicio individualmente.

Lección 19 — Access logs de Envoy

Revisar el access log del sidecar ayuda a diferenciar si un problema es de red/mesh o del propio código de la aplicación.

Lección 20 — Dashboards de Grafana para Istio

Los dashboards predefinidos ya cubren las métricas más comunes del mesh, dando un punto de partida sólido antes de personalizar según necesidades específicas.

Módulo 5: Resiliencia y escalado

Lección 21 — Timeouts y retries avanzados

Sin timeout configurado, peticiones a un servicio lento podrían quedar esperando indefinidamente, agotando recursos en cascada.

```
http: - timeout: 5s
```

Lección 22 — Fault injection — chaos testing

Simular fallos deliberados en pruebas descubre debilidades en un entorno controlado, en vez de encontrarlas por primera vez durante un incidente real.

```
fault: delay: fixedDelay: 5s
```

Lección 23 — Rate limiting con Istio

Limita cuánto tráfico procesa un servicio en un período dado, protegiendo su disponibilidad ante un ataque de volumen masivo o un exceso accidental.

Lección 24 — Multi-cluster mesh

Permite que servicios en clusters distintos se comuniquen como parte del mismo mesh lógico, dando resiliencia geográfica real entre regiones.

Lección 25 — Escalado del control plane (istiod)

Un mesh con miles de sidecars distribuye la carga de gestionar configuración entre varias réplicas de istiod, evitando un cuello de botella.

```
kubectl scale deployment istiod --replicas=3
```


Módulo 6: Operación avanzada

Lección 26 — Actualización de Istio sin downtime

Migrar namespaces gradualmente a una nueva revisión de istiod detecta problemas antes de afectar a todo el mesh de producción a la vez.

Lección 27 — Troubleshooting de sidecars y Envoy

'istioctl proxy-config' confirma qué configuración realmente se propagó a un proxy específico, que podría diferir de lo definido si hay un problema de propagación.

```
istioctl proxy-config routes mi-pod
```

Lección 28 — Rendimiento — overhead del sidecar

Medir el overhead con pruebas de carga propias es más confiable que confiar en benchmarks genéricos, ya que depende de las características específicas del tráfico.

Lección 29 — Migración progresiva a Istio (brownfield)

Empezar la migración con servicios menos críticos permite aprender y ajustar el proceso antes de arriesgar los servicios más importantes de producción.

Lección 30 — Proyecto final: service mesh completo listo para producción

Se integra todo lo anterior: mTLS STRICT, AuthorizationPolicy, traffic management, observabilidad con Kiali/Grafana/tracing y escalado de istiod — el nivel esperado de un mesh listo para producción real.